

El desarrollo de los dos modelos de aprendizaje automático siguió el marco **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), el proceso estándar para proyectos de minería de datos y aprendizaje automático. Se eligió por ser sistemático, iterativo y orientado a la evaluación objetiva, y porque sus fases mapean de forma natural el trabajo realizado —desde la comprensión del problema hasta el despliegue en producción. Este anexo describe cada fase y remite a los demás anexos para el detalle técnico.

A.1 Resumen de fases y artefactos

Tabla A.1. Fases de CRISP-DM y su correspondencia con el trabajo realizado.

Fase CRISP-DM	Actividad en el proyecto	Artefacto / evidencia
1. Comprensión del problema	Objetivos y alcance definidos en la introducción: clasificar issues (tipo+severidad) y resumir commits	Introducción; diagrama de Ishikawa
2. Comprensión de los datos	Selección y análisis exploratorio de datasets públicos (Kaggle, MSR2013, NNGen/MCMD)	Apéndice B (§B.1, §B.2, §B.3)
3. Preparación de los datos	Limpieza, tokenización, mapeos, partición estratificada 80/10/10 (semilla 42)	Apéndice B
4. Modelado	TF-IDF + clasificadores lineales; seq2seq pointer-generator desde cero	Apéndice B; scripts de training/
5. Evaluación	F1-score macro, BLEU-4, ROUGE-L; compuerta de calidad en el pipeline	Capítulo 09; Apéndice B; Apéndice D
6. Despliegue	Microservicios FastAPI, pipeline MLOps (Airflow), infraestructura ECS (CDK), integración en el frontend	Apéndices C, D, E, F

Nota. Fuente: Elaboración propia, 2026.

A.2 Fase 1 — Comprensión del problema

Se identificó que, en la plataforma Mini-GitHub, la clasificación de issues y la documentación de commits se realizan manualmente, de forma inconsistente. Se definieron dos objetivos de negocio: (i) asistir el *triage* de incidencias sugiriendo tipo y severidad, y (ii) asistir la documentación de cambios sugiriendo el mensaje de commit. Ambos como **sugerencias editables**, no decisiones automáticas. Restricción central del proyecto: entrenar modelos propios, **sin modelos de lenguaje de gran escala de terceros**, con inferencia en CPU.

A.3 Fase 2 — Comprensión de los datos

Se seleccionaron conjuntos de datos públicos alineados con la bibliografía del proyecto: *GitHub Bugs Prediction* (Kaggle) para el tipo, el *Eclipse and Mozilla Defect Tracking Dataset* (Lamkanfi et al., MSR 2013) para la severidad, y el corpus **NNGen** (Liu et al., 2018) ampliado con **MCMD** para el resumen de commits. El análisis exploratorio evidenció el fuerte desbalance de clases (la clase `question` es minoritaria; la severidad `media / normal` domina ~75 % del corpus de Bugzilla), hecho determinante para las decisiones de modelado (§A.5).

A.4 Fase 3 — Preparación de los datos

- **Texto de issues:** concatenación de título y cuerpo; sustitución de bloques de código por el token `CODE` y de URLs por `URL`; eliminación de HTML; minúsculas; descarte de textos < 10 caracteres.
- **Severidad:** mapeo de las siete severidades de Bugzilla a los cuatro niveles del proyecto (blocker/critical → crítica, mayor → alta, normal → media, minor/trivial → baja; enhancement excluido).
- **Commits:** corpus pre-tokenizado; truncado del diff a 100 tokens y del mensaje a 25; vocabularios contruidos solo con el conjunto de entrenamiento (frecuencia mínima 2).
- **Partición:** estratificada 80/10/10 (train/validación/prueba) con **semilla 42** en todos los casos, para garantizar reproducibilidad.

A.5 Fase 4 — Modelado

- **Clasificación:** vectorización TF-IDF (100 000 rasgos, unigramas y bigramas) y comparación de Regresión Logística frente a SVM lineal, ambos con `class_weight='balanced'`, seleccionando el de mayor F1-macro en validación. Elección de modelos lineales por su robustez sobre representaciones dispersas de alta dimensión y su idoneidad para inferencia en CPU.
- **Resumen:** modelo secuencia a secuencia entrenado **desde cero** (codificador GRU bidireccional + decodificador con atención de Luong + mecanismo *pointer-generator*), en la línea de la traducción automática neuronal aplicada a mensajes de commit.

A.6 Fase 5 — Evaluación

Se evaluó cada modelo una única vez sobre el conjunto de prueba, tras la selección por validación. Métricas: **F1-score macro** y métricas por clase (precisión/recall/F1) para la clasificación; **BLEU-4** y **ROUGE-L** para el resumen. La evaluación se integró además en el pipeline como una **compuerta de calidad** que sólo promueve un modelo si supera el umbral ($F1 \geq 0,60$ tipo, $F1 \geq 0,40$ severidad, $BLEU-4 \geq 5,0$ resumen). Resultados completos en el Capítulo 09 y en el Apéndice B.

A.7 Fase 6 — Despliegue

El despliegue —fase frecuentemente omitida en trabajos académicos— es una parte central de este proyecto por su orientación DevOps. Comprende: el encapsulamiento de cada modelo como microservicio FastAPI (Apéndice C), la automatización del ciclo de vida con un pipeline MLOps de Airflow (Apéndice D), la integración en el frontend como sugerencias editables (Apéndice E) y el despliegue de la plataforma completa en AWS ECS Fargate mediante infraestructura como código (Apéndice D), con evidencia de funcionamiento en producción (Apéndice F).